Blueprint Hackathon · Version 2

Team Brief: What We Build, and Who Builds What

A plan built backwards from the judging rubric, with the work split down the middle. Read all of it before we write any code — the scope is deliberately small, and section 6 explains why that is the whole strategy.

Jul 31	6 days	54 pts	24 / 24
Deadline, 23:00 GMT+2	Time remaining	Maximum score	Points split between us

1. The read on this competition

Two things on the event page decide our strategy. First, one of the two judges is a Senior Application Security Analyst at Corteva Agriscience — a professional who can tell careful security work apart from a demo that merely looks like security work. Our project gets read properly instead of skimmed, which cuts both ways.

Second, and more important: do the rubric arithmetic. Innovation is worth 4 points out of 54. Documentation, testing, accessibility and deployment are worth 16 combined, and they are exactly where most submissions in a beginner-friendly, all-ages bracket score zero or two. We do not win with the cleverest idea. We win by being the only entry that is finished, tested, deployed, documented and accessible.

> **THE OPERATING PRINCIPLE**
> Judge for craft, not ambition. Every hour spent widening scope costs points in four categories at once, because an unfinished feature breaks Feature Completeness, Testing, Deployment and User Flow simultaneously. Either of us can veto a new feature at any point, no discussion needed.

2. The project

Working title to be agreed — shortlist: Domainscope, Netgrade, Clearsight, Frontdoor

Enter a domain. Get a scored security report that a non-technical business owner can act on. Then three things that turn a scanner into a product:

- A plain-language audio briefing. The report speaks its top three risks aloud — “your biggest risk is a missing DMARC record; anyone can send email pretending to be you; fix this first.” Generated with ElevenLabs, cached per scan.
- Side-by-side comparison. Scan your domain against a competitor’s. This reframes the product from a technical readout into a business question an owner actually has.
- Live re-scan. Fix something, scan again, watch the grade move. Visible cause and effect is disproportionately memorable on camera.

Every check is passive and read-only

The same class of inspection SSL Labs and securityheaders.com perform against any public host. No active exploitation, no authenticated scanning, nothing against a host we do not own. This is the ethical line and, given who is judging, also a scoring advantage.

3. Scope — locked on day one

Eight checks, all finished, beats twenty half-working ones.

Check	What it inspects	Why a business cares
Email spoofing	SPF, DKIM and DMARC records and policy strength	Missing DMARC means anyone can send mail as you
TLS configuration	Protocol versions, cipher strength, certificate expiry	Expired certs break trust and browser access
Security headers	HSTS, CSP, X-Frame-Options, X-Content-Type-Options	Cheapest defence against a whole class of web attacks
Cookie flags	Secure, HttpOnly and SameSite attributes on set cookies	Weak flags expose user sessions to theft
Exposed artefacts	Known paths such as .git, .env and .DS_Store	Leaked credentials and source are a direct breach path
DNS hygiene	Nameserver spread, zone transfer exposure, dangling records	Dangling records enable subdomain takeover
Certificate history	Certificate transparency logs for the domain	Reveals forgotten subdomains still exposed
Scored report	Weighted grade with prioritised plain-language remediation	Turns findings into a decision, not a data dump

This list goes in the README on day one and does not grow. Feature Completeness only pays 4 out of 4 if everything we listed is done — so we list less, and finish all of it.

4. Stack and architecture

Layer	Choice	Reasoning
Backend	Python, FastAPI, asyncio	DNS and TLS work is far easier in Python; async gives concurrent checks for free
Frontend	Server-rendered templates, minimal JS	Fewer moving parts, faster to make accessible, nothing to hydrate
Checks	One module per check, uniform interface	Modularity is directly scored under System Architecture
Audio	ElevenLabs, generated then cached per scan	Never call the API live during the recorded demo
Cache	Per-domain results with a TTL	Repeat scans are instant; carries the scalability story
Tests	pytest, unit plus integration	A full 4 points most teams leave on the table
Deploy	Docker, single container, Fly.io or Railway	One repo, one deploy, one live URL — clean and repeatable

5. Rubric coverage — all twelve categories, and who owns each

If the artefact named here does not exist by submission day, that row is not done.

Category	Pts	Owner	What we ship to earn it
Innovation & Creativity	4	Mike	Findings translated into prioritised business actions plus a spoken briefing

Category	Pts	Owner	What we ship to earn it
Technical Complexity	4	Mike	Concurrent async checks across seven protocols, with parsing and weighted scoring
Scalability	4	Mike	Async fan-out, TTL cache, rate limiting, stateless container, written Scaling section
Code Quality	4	Mike	One module per check, uniform interface, type hints, linted, consistent naming
System Architecture	4	Mike	Clear separation: check modules, orchestrator, scoring, presentation
Feature Completeness	4	Mike	The eight checks in section 3, all working, nothing else promised
Interface Design	4	Teammate	Responsive, calm, readable; severity legible at a glance without clutter
Accessibility	4	Teammate	Semantic HTML, keyboard nav, focus states, ARIA, contrast, never colour alone
User Flow	4	Teammate	One input, one action, one report. No dead ends, no guesswork
Documentation	4	Teammate	README with Mermaid diagram, setup, decisions-and-tradeoffs, threat-model note
Testing	4	Teammate	Unit tests per module, integration test on the orchestrator, edge cases
Deployment	4	Teammate	Live public URL, one-command local setup, documented deploy
Video (3 segments)	6	Both	Cold open, demo, technical explanation — see section 8

6. The work split — 24 points each

This is the section that matters most. Read it together and agree it out loud.

The instinct in a mixed-experience team is for the security person to build and the other person to help. That wastes half the team and costs us points, because the six categories a non-security teammate can own outright are worth exactly as much as the six that need security knowledge — 24 points each. This is a genuine fifty-fifty split, not a polite one.

Mike — the engine

Needs security knowledge

- Eight check modules, one file each, uniform interface
- Async orchestrator with per-check timeouts and concurrency limits
- Risk scoring and severity weighting
- Plain-language explanation and fix text for every finding
- Caching layer and rate limiting
- Threat model and the Limitations section
- Technical segment of the video

Owns: Innovation, Technical Complexity, Scalability, Code Quality, System Architecture, Feature Completeness — 24 pts

Teammate — the product

No security knowledge required

- Report interface, comparison view and full visual identity
- Accessibility: keyboard nav, focus states, contrast, ARIA, axe clean
- User flow from input through report to comparison
- README, architecture diagram, setup instructions
- pytest scaffolding, edge cases, continuous integration
- Docker, deployment, live URL, one-command local setup
- ElevenLabs audio integration, demo script and video edit

Owns: Interface Design, Accessibility, User Flow, Documentation, Testing, Deployment — 24 pts

Shared, and decided together

- The comparison feature. Mike's engine runs two scans concurrently; the teammate's interface presents them side by side. Neither half works alone.
- Scope discipline. Either of us can veto a new feature. No justification required.
- The submission itself. Both of us check the Devpost form before it goes in.

7. The contract — agree this in hour one

This is the single decision that lets both of us work without ever waiting on the other.

Before either of us writes real code, we agree one JSON shape for a scan result. Then Mike writes a mock file full of fake findings and hands it over. From that moment the teammate builds the entire frontend against the mock and is never blocked by the engine being unfinished — and Mike never has to wait for a UI to test a check.

```
{
  "domain": "example.nl",
  "scanned_at": "2026-07-28T14:03:11Z",
  "grade": "C",
  "score": 61,
  "checks": [
    {
      "id": "email_spoofing",
      "title": "Email spoofing protection",
      "status": "fail",              // pass | warn | fail | error
      "severity": "high",            // critical | high | medium | low | info
      "summary": "No DMARC policy is published.",
      "explanation": "Anyone can send email that appears to come from your domain.",
      "fix": "Publish a DMARC record starting at p=none, then tighten to p=reject.",
      "evidence": { "dmarc_record": null, "spf_record": "v=spf1 include:..." }
    }
  ]
}
```

- Frozen means frozen. Any change to this shape after day one gets announced before it is committed, because it breaks the other person's work silently.
- Every check returns this shape, including on failure. An unreachable domain returns status `error`, never an exception the frontend has to handle.
- The mock file stays in the repo as a test fixture. It is what the integration tests run against, so it earns points twice.

8. Six days, running in parallel

Two columns because we work simultaneously, not in sequence.

Day	Mike — engine	Teammate — product	Done when
1	Agree contract, write mock, build checks 1–4	Layout, report view and visual identity against the mock	Four checks return real data; a mock report renders in a browser
2	Checks 5–8 and the async orchestrator	Comparison view, responsive pass, empty and error states	All eight checks run concurrently; every UI state has a design
3	Scoring, severity weighting, remediation text	Accessibility pass, keyboard nav, contrast, axe clean	A scan produces a grade; axe reports zero violations
4	Integration — wire the real engine to the interface	Docker, deploy, live URL, README and architecture diagram	The live URL scans a real domain end to end

Day	Mike — engine	Teammate — product	Done when
5	Threat model, Limitations section, technical script	Audio briefing, test suite green, record and edit the video	Video uploaded; repository documented; tests passing
6	Buffer — rehearse the demo, fix nothing new	Buffer — submit early, verify the Devpost form	Submitted with hours to spare, not minutes

THE ONE DEPENDENCY

Day 4 is the only real integration point, and it is low-risk precisely because the contract was frozen on day 1. If integration slips past day 4, cut a check rather than cutting tests, docs or accessibility — seven working checks with full craft scores far higher than eight with none.

9. If one of us falls behind

Situation	What we do
Engine is behind on day 3	Cut to six checks. Feature Completeness is scored against what the README promises, so edit the README instead of shipping something broken.
Interface is behind on day 3	Mike stops adding checks and takes the report template. The engine is worth nothing without a surface to show it.
Deployment fails on day 4	Both of us drop everything. A local-only project caps Deployment at 0 and undermines the demo. This is the highest-priority failure on the list.
Audio briefing is not working by day 5	Cut it. It is one innovation beat, not a scored category, and it is the cheapest thing to lose.
We are both behind on day 5	Record the video anyway with what exists. A missing video costs 6 points plus the judges' entire impression of the project.

10. Video — three separate scores, hit each deliberately

Six rubric points, but effectively all of our memorability. Two judges are working through dozens of submissions, and the first ten seconds decide whether we get real attention. No introductions — cold open on the product doing something. A domain gets typed in, results land, something bad is visible on screen. We introduce ourselves after they care.

Segment	Length	What it must contain
Cold open	~10 sec	A real scan on a real domain, finding something genuinely true. No titles, no names, no logo animation.
Problem	~30 sec	Small businesses cannot see their own public attack surface and cannot afford an audit
Demo	~2 min	Full scan, the audio briefing playing aloud, side-by-side comparison, then a live fix and re-scan with the grade moving
Technical	~1 min	Stack, architecture, the concurrency and scoring decisions, and why we made them
Close	~20 sec	Who it is for, the live URL, and what it deliberately does not do

– Scan a real, recognisable domain. Not example.com, not a site we broke on purpose. A judge who watches the tool surface a genuine missing DMARC policy remembers it; nobody remembers a scan of a deliberately vulnerable test site.

- Our own voice, not a synthetic one. An AI voiceover reads as low-effort and strips out the thing that makes a submission memorable — that a person made it. The synthetic voice belongs inside the product, not narrating the video.
- Problem, Demo and Technical are scored independently at 2 points each. A great demo with no technical explanation still loses a third of the video marks.

11. The Limitations section

Small, cheap, and the highest-leverage paragraph in the repository.

The README gets a section titled What this tool deliberately does not do, and why. No active exploitation. No authenticated scanning. No claims about internal network posture. An explicit note that a passing grade is not a clean bill of health — it means these specific public checks passed on this date.

Almost no student submission admits what it cannot do. To a working security professional this reads as genuine maturity rather than as weakness, and it is exactly the kind of thing that gets mentioned out loud when two judges are comparing entries.

12. Hard rules

- Fresh repository. First commit inside the event window. Nothing lifted from any existing project, portfolio scaffold or template either of us already owns.
- The 10-point deduction is the biggest single swing in the rubric — larger than any whole category. Not worth any shortcut it could buy.
- AI is explicitly allowed. Claude Code and similar tools are fair game, but every line must be generated during the window. Corollary: we must both be able to explain our own half on camera. If we cannot account for a decision, an application security judge will notice.
- No plagiarism. Open-source libraries and frameworks are fine; core logic must be ours.
- Passive checks only. No active exploitation against any host we do not own.
- Submit on day 5 if we can. Early submissions get looked at by fresher judges. Unglamorous, and real.

Blueprint Hackathon team brief, version 2 · prepared 25 July 2026 · submission deadline 31 July 2026, 23:00 GMT+2